SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5102
NAME:PREETHI R
REG NO:192511352

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I __ Preethi R(192511352)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

Introduction

Secure file transfer is an important part of modern computer networks because organizations frequently exchange confidential documents, images, videos, databases, and other digital files. These files must be transferred efficiently while protecting them against unauthorized access, modification, and data corruption.

An SFTP-like system is a file transfer application that simulates the secure transfer of files between a sender and a receiver using encryption, packet handling, and integrity verification. However, a poorly designed implementation may experience slow transfer speeds, synchronization errors during encryption and decryption, packet loss or duplication, and corrupted output files, especially when transferring large files of 100 MB or more.

The main objective of optimizing this system is to correct encryption and decryption errors, improve packet handling, ensure data integrity, and increase transfer throughput without weakening security.

1. Debugging Encryption and Decryption Synchronization Issues

Encryption converts readable data (plaintext) into an unreadable form (ciphertext) to protect it from unauthorized access. Decryption converts the ciphertext back into the original data using the appropriate key and cryptographic parameters.

In a secure file transfer system, the sender encrypts the file before transmission, and the receiver decrypts the received data. Both sides must follow the same cryptographic protocol and correctly manage keys, nonces, initialization vectors, and authentication tags.

Common synchronization problems

a) Incorrect encryption and decryption parameters

Encryption and decryption must use compatible algorithms, keys, nonces, and other required parameters. If the receiver uses an incorrect key or nonce, decryption may fail or produce invalid data.

Solution: Use a well-established authenticated encryption algorithm, such as AES-GCM, through a trusted cryptographic library. The protocol must clearly define how encryption parameters are generated, transmitted, and validated. Nonces must never be reused with the same key in ways prohibited by the algorithm.

b) Incorrect handling of encryption state

Some encryption modes maintain state across blocks or chunks. If the sender and receiver process data in an inconsistent order, or if the implementation incorrectly resets or reuses state, the recovered data may be corrupted.

Solution: Use a documented encryption interface and follow its requirements for each message or chunk. For independent chunks, use a unique nonce for each chunk and authenticate its metadata. Do not reset or reuse encryption state incorrectly.

c) Buffer-size mismatch

The sender may read a file in fixed-size chunks, while the receiver expects a different chunk size or assumes that every packet has the same length. This can cause missing bytes, truncated data, or incorrect reconstruction of the original file.

Solution: Include explicit length information in each packet or frame. The receiver should read the complete header, determine the expected payload length, and then read exactly that many bytes before processing the packet.

d) Incorrect packet ordering

When multiple packets are transferred concurrently, they may arrive in a different order from the order in which they were sent. If the receiver writes them immediately without checking their sequence, the output file may contain incorrectly ordered data.

Solution: Assign a sequence number or chunk index to each data packet. The receiver should place each chunk at its correct file offset or reorder chunks before final reconstruction.

Example

Suppose a 100 MB file is divided into smaller encrypted chunks. If chunk 5 is decrypted using the wrong nonce or is written before chunk 4, the receiver may produce a corrupted file.

To avoid this problem, each chunk should contain authenticated metadata identifying its transfer session, sequence number, and expected length. The receiver must validate the metadata and authenticate the ciphertext before writing the recovered plaintext.

Therefore, correct synchronization requires consistent cryptographic parameters, reliable chunk boundaries, and proper coordination between the sender and receiver.

2. Fixing Packet Handling and Integrity Verification Errors

Packet handling is the process of dividing file data into smaller units, transmitting those units, and reconstructing the original file at the receiver. Correct packet handling is essential for preventing data loss, duplication, reordering, and corruption.

Common packet-handling errors

a) Missing or incomplete packets

During transmission, a packet may be lost because of a network failure or interrupted connection. If the receiver assumes that all packets have arrived, the reconstructed file may be incomplete.

Solution: Maintain a record of the chunks transmitted and received. Use acknowledgements, timeouts, and retransmission where necessary. Missing chunks should be retransmitted rather than silently ignored.

b) Duplicate packets

A sender may retransmit a packet after failing to receive an acknowledgement, even though the original packet has already arrived. If the receiver writes the same data twice, the output file may become corrupted.

Solution: Use unique chunk identifiers or sequence numbers. The receiver should detect duplicate chunks and avoid processing or writing the same chunk more than once.

c) Incorrect packet boundaries

Network protocols generally transmit streams of bytes, not complete application-level messages. A single read operation may return only part of a packet, or it may return data spanning multiple packets.

Solution: Implement explicit framing using a fixed-size header or a length-prefixed format. The receiver must use a loop to read the complete header and payload instead of assuming that one read operation receives the entire packet.

d) Incorrect error handling

If the sender or receiver fails to check return values, timeouts, or connection errors, transmission failures may go unnoticed.

Solution: Validate packet lengths, handle connection interruptions, apply suitable timeouts, and report failures clearly. The receiver should not mark a transfer as successful until all required data has been received and verified.

Integrity verification

Integrity verification ensures that the received data matches the data sent by the sender. A file may be corrupted because of software bugs, transmission errors, incorrect buffer management, or malicious modification.

A secure implementation should use **authenticated encryption**, such as AES-GCM or ChaCha20-Poly1305, to provide confidentiality and cryptographic integrity. These algorithms generate an authentication tag that the receiver verifies before accepting the decrypted data.

For each encrypted chunk, the sender should authenticate relevant metadata, such as the transfer identifier, chunk number, and expected length. The receiver must verify the authentication tag before accepting the plaintext.

A cryptographic hash, such as SHA-256, may also be calculated over the completed source file and compared with a trusted expected hash to detect accidental corruption or confirm that the reconstructed file matches the source. However, a plain hash sent alongside the file without authentication does not protect against an attacker who can modify both the file and its hash.

Steps for reliable integrity verification

1. Calculate the source file's size and, when required, its SHA-256 hash.
2. Divide the file into chunks and encrypt each chunk using authenticated encryption.
3. Attach the necessary packet metadata and authentication information.
4. Transmit the encrypted chunks through the network.
5. Check the packet length, transfer identifier, and chunk sequence at the receiver.
6. Verify the authentication tag before accepting each chunk.
7. Reassemble the chunks in the correct order.
8. Verify the final file size and compare the final hash with a trusted expected value when available.

These measures help ensure that incomplete, duplicated, reordered, or modified data is detected rather than silently accepted.

3. Optimizing Throughput for Large File Transfers (≥100 MB)

Throughput refers to the amount of data successfully transferred per unit of time. When transferring files of 100 MB or more, inefficient encryption, repeated disk operations, excessive network communication, and unnecessary memory copying can reduce performance.

The goal is to improve throughput while preserving confidentiality, integrity, and reliable file reconstruction.

a) Use chunk-based file processing

Instead of loading the entire file into memory, divide it into manageable chunks, such as 256 KB, 1 MB, or 4 MB, depending on the system and network conditions.

The sender reads and encrypts one chunk at a time, while the receiver authenticates, decrypts, and writes each chunk. This reduces peak memory usage and supports transfers of very large files.

Chunk size should be tested under realistic conditions. Very small chunks increase per-packet and per-operation overhead, while excessively large chunks may increase memory usage and make recovery from failures less efficient.

b) Use efficient cryptographic libraries

Encryption and decryption should use established, optimized cryptographic libraries rather than custom encryption code. Libraries such as Python's cryptography package provide implementations of modern authenticated encryption algorithms.

AES-GCM may benefit from hardware acceleration on supported processors. ChaCha20-Poly1305 is another suitable authenticated encryption option and may perform well on systems without dedicated AES acceleration.

The appropriate algorithm should be selected based on security requirements, platform support, and measured performance.

c) Reduce unnecessary disk operations

Repeatedly opening and closing files or writing very small amounts of data can reduce performance.

The sender should keep the source file open during the transfer and use buffered reads. The receiver should use buffered writes and avoid unnecessary copying of data between memory buffers.

For reliable transfers, write incoming data to a temporary file and rename it to the final filename only after successful verification. This prevents an incomplete transfer from being mistaken for a complete file.

d) Use controlled concurrency

When the system supports it, multiple chunks can be processed concurrently to improve utilization of CPU, storage, and network resources.

For example, a producer-consumer design can use a bounded queue: one task reads file chunks, worker tasks encrypt or decrypt them, and another task sends or writes the results.

However, concurrency must be implemented carefully. Nonces must remain unique under each encryption key, chunk identifiers must be preserved, and results must be written in the correct order or at the correct offsets. The number of workers should be limited to prevent excessive memory usage and resource contention.

e) Reduce network overhead

Each packet or application-level frame adds some overhead. Using appropriately sized chunks can reduce the number of framing operations and acknowledgements required for a large transfer.

Where appropriate, persistent connections can avoid repeatedly establishing connections for each chunk. Timeouts and retransmission policies should also be configured according to the expected network conditions.

f) Support resumable transfers

If a 100 MB or larger transfer is interrupted, restarting from the beginning wastes time and bandwidth.

The system can maintain a transfer identifier and a record of successfully received, authenticated chunks. After reconnection, the sender retransmits only the missing or unverified chunks.

The receiver must validate the identity of the transfer and ensure that previously accepted chunks belong to the same file and session. Resumption must not permit data from different transfers to be mixed together.

g) Measure performance systematically

The system should be tested with files of different sizes, including files of at least 100 MB. Measure the total transfer time, effective throughput, CPU usage, peak memory consumption, and number of retransmissions.

Throughput can be calculated as:

Throughput = Total file size (bits) ÷ Transfer time (seconds)

For example, transferring a 100 MB file in 20 seconds gives an approximate throughput of 40 megabits per second when 100 MB is interpreted as 800 megabits.

Optimization should be based on repeated measurements under comparable network and hardware conditions. Security checks should never be removed merely to increase transfer speed.

4. Comparison with Insecure File Transfer Methods and Justification of Improvements

Traditional insecure file transfer methods, such as FTP without TLS, do not provide the same protections as a properly configured secure transfer system. In basic FTP, credentials and file contents may be transmitted without encryption, making them vulnerable to interception.

An optimized secure file transfer system improves confidentiality, integrity, and reliability while reducing unnecessary processing overhead.

Feature	**Insecure FTP**	**Optimized secure transfer system**
Confidentiality	File contents may be transmitted in plaintext.	Authenticated encryption protects file contents.

Authentication	Basic FTP credentials may be exposed during transmission.	Uses authenticated secure connections and appropriate identity verification.
Integrity	Does not inherently provide cryptographic protection against file modification.	Authenticated encryption detects unauthorized modification of protected data.
Packet handling	Reliability depends on the underlying protocol and implementation.	Uses framing, chunk identifiers, validation, and recovery mechanisms.
Large-file performance	Performance depends on network, storage, and implementation.	Chunking, buffering, and controlled concurrency can improve throughput.
Recovery from interruption	Depends on the client, server, and protocol features.	Can support verified chunk-level resumption.
Memory usage	Depends on the implementation.	Chunk-based processing avoids loading the entire file into memory.
Security against interception	Weak when data and credentials are unencrypted.	Encryption protects data in transit when configured correctly.

Justification for the improvements

The optimized system is preferable because it combines security with efficient data processing. Authenticated encryption protects confidentiality and detects unauthorized changes. Correct packet framing, sequence tracking, and retransmission reduce the risk of corrupted or incomplete files. Chunk-based processing limits memory consumption, while buffering and controlled concurrency can improve throughput for large transfers.

However, the actual speed improvement depends on the network, processor, storage device, encryption algorithm, and implementation. Performance claims should therefore be supported by benchmark results rather than assumed in advance.

For production systems, using a well-maintained implementation of SFTP over SSH or another established secure transfer protocol is generally safer than developing a custom protocol. A

custom SFTP-like implementation should be treated as a simulation or educational project unless it has undergone appropriate security review and testing.

Conclusion

Optimization of a secure file transfer system requires correcting encryption and decryption synchronization errors, improving packet handling, implementing reliable integrity verification, and reducing unnecessary processing overhead.

Using authenticated encryption, explicit packet framing, unique chunk identifiers, buffered I/O, controlled concurrency, and resumable transfers can improve reliability and throughput for files of 100 MB or larger. Compared with insecure file transfer methods, the optimized system provides stronger confidentiality and integrity protections while supporting more efficient data transmission.

Therefore, the most effective solution is to combine established cryptographic algorithms with reliable transfer mechanisms and systematic performance testing. This ensures that large files can be transferred efficiently without compromising the security or correctness of the received data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation